CSC 1102: Structured and Object-Oriented Programming

Practical Exam Project

STUDENT MANAGEMENT SYSTEM

Group Details

No.	Name	Registration Number	Role
1	LOMER FRANCIS PETER	25/U/26198/EVE	Member
2	MAYANJA JUDITH	25/U/28752/EVE	Member
3	**AKELLO SUNDAY OBONYO**	**24/U/03151/EVE**	**Group Leader**
4	MADETE HECTOR	24/U/25933/EVE	Member
5	ANYINE ELIZABETH	25/U/29497/EVE	Member

GitHub Repository

<PASTE YOUR GROUP'S GITHUB LINK HERE>

Submitted to: Dr. Lillian Muyama

1. Description of the Application

The **Student Management System** is a desktop application written in Java with a Swing graphical user interface. It helps a university administrator keep a register of students, record each student's marks in three subjects, and automatically work out the total, average, letter grade, remark and pass/fail status. Courses are managed in a separate tab, and every student is linked to a course. All data is held in memory in arrays, so no database is required. A few sample students and courses are loaded at start-up so the system can be demonstrated immediately.

2. Main Features

- **Register students** (registration number, name, age, course and three marks) with full input validation and duplicate checking.
- **Update and delete** students: click a row in the table to load it into the form.
- **Automatic results:** total, average, grade (A to F), remark and PASS/FAIL (pass mark 50) for every student.
- **Search** by registration number, name or course, and **sort** by average (highest first) or by name (A to Z).
- **Statistics:** class average, highest and lowest average, number passed/failed and grade distribution.
- **Manage courses:** add and delete courses; the Courses tab shows how many students are enrolled. A course with enrolled students cannot be deleted.

3. Classes and Their Purpose

Class	Purpose	Key members
Main	Entry point. Holds the group details as comments and starts the GUI.	main()
Student	Entity class: one student and his/her marks.	private regNo, name, age, courseCode, double[] marks; 2 constructors; getters/setters; overloaded setMarks(); getTotal(), getAverage(), getGrade(), getRemark(), getStatus()
Course	Entity class: one course unit.	private courseCode, courseName, creditUnits, lecturer; 2 constructors; getters/setters; toString()
StudentManager	Stores and manages the data in **Student[]** and **Course[]** arrays; contains the application logic.	addStudent(), findStudent(), updateStudent(), deleteStudent(), overloaded searchStudents(), sortByAverage(), sortByName(), getStatistics(), addCourse(), deleteCourse(), loadSampleData()
Validator	Utility class that checks and cleans user input.	private constructor; static cleanText(), isWholeNumber(), isDecimalNumber(), isValidName(), isInRange()
StudentManagementGUI	Builds the Swing window (tabs, forms, tables, buttons) and handles user events.	buildInterface(), readStudentForm(), addStudent(), updateStudent(), deleteStudent(), searchStudents(), addCourse(), showStudents()

4. How the Classes Interact

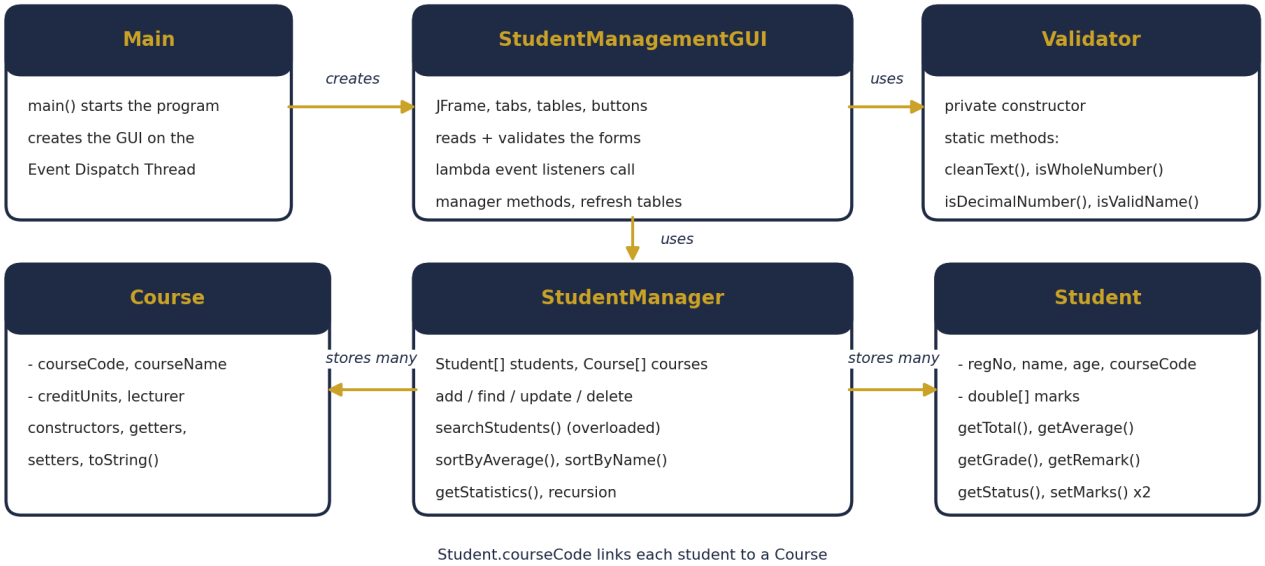


Figure 1: Class interaction diagram

Main creates the **StudentManagementGUI**. The GUI creates one **StudentManager** and uses it for every operation. When the user fills in the form, the GUI first cleans and checks each value with **Validator**, then builds a **Student** object and passes it to the manager, which stores it in its **Student[]** array. Courses work the same way with **Course** and the **Course[]** array. Each Student keeps the code of its Course, so the manager can count the students enrolled in a course. The GUI never touches the arrays directly: it only calls manager methods and then redraws its tables.

5. How the Exam Requirements Are Met

Requirement	Where it is demonstrated
Object-oriented design (3+ classes)	Student, Course, StudentManager, Validator and StudentManagementGUI (plus Main) - each with a clear role; constructors (default, parameterized, constructor chaining with this()), private attributes, public getters/setters, objects created with new.
Graphical user interface	Swing: JFrame, JTabbedPane, JTable, JTextField, JComboBox, JButton, JOptionPane dialogs; events handled with lambda ActionListeners.
Flow control	if / else-if (grade calculation, validation), switch (remark), for (arrays, totals), while (Validator.isWholeNumber), do-while (Validator.cleanText), enhanced for (grade distribution), and the ? : conditional operator.
Arrays	Student[] and Course[] in StudentManager (add, search, delete with shifting, bubble-sort); double[] marks inside Student; String[] and 2-D double[][] for the sample data.
Other course concepts	Method overloading (searchStudents, setMarks), recursion (sumOfAverages), static members and constants, encapsulation (getMarks() returns a copy), private constructor in a utility class.

6. Screenshots

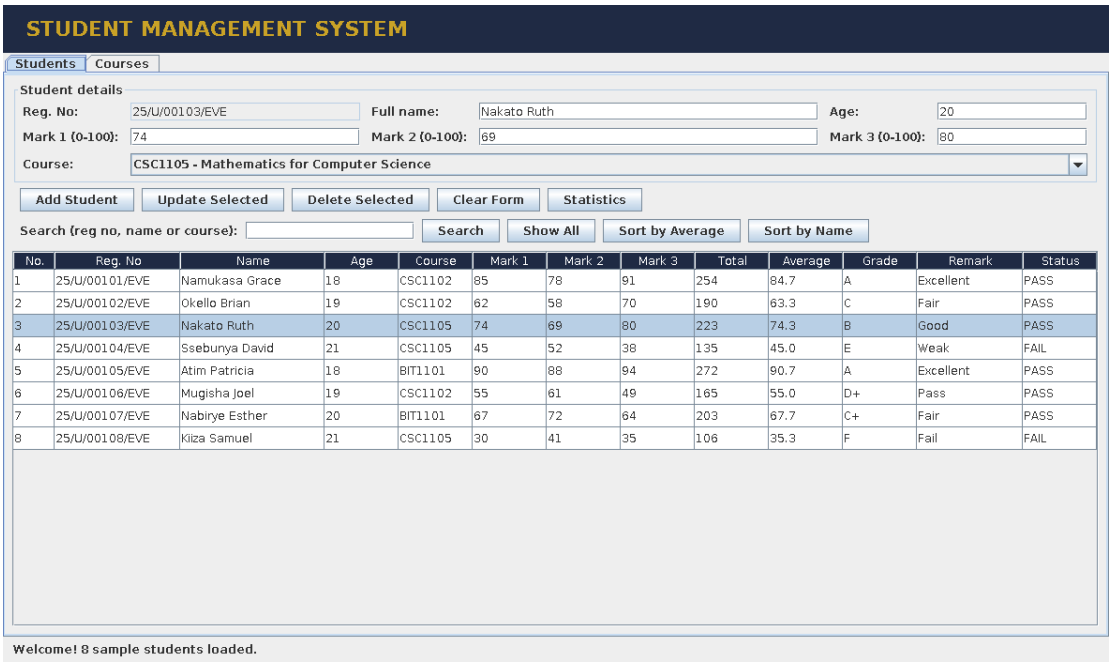


Figure 2: Students tab - a row is selected and loaded into the form

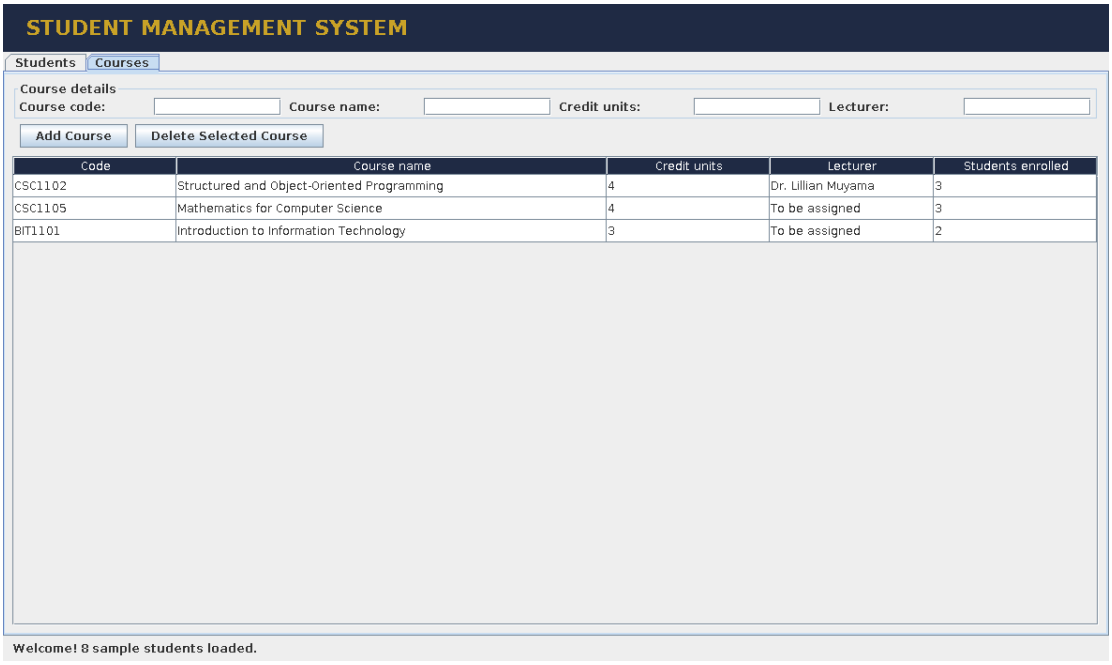


Figure 3: Courses tab - course list with the number of enrolled students

7. How to Run the Program

Requires JDK 17 or later (JDK 21 recommended). Open a terminal in the project folder and run:

```
javac *.java
java Main
```

Alternatively, open the folder in VS Code (with the Extension Pack for Java) and run **Main.java**.